

Checklist:

Team Topologies – Static Team Topologies

1. Intentional Team Design

- Are teams designed intentionally rather than formed ad hoc?
- Are team boundaries aligned to business domains or value streams?
- Are teams long-lived enough to build ownership and context?
- Are teams stable, rather than frequently shuffled between projects?
- Have we avoided creating teams only as a reaction to short-term problems?

2. Flow of Change

- Can work move from idea to production with minimal handoffs?
- Are teams organized to reduce wait times between development, testing, operations, and business-as-usual work?
- Do teams receive fast feedback from production/live systems?
- Are software delivery teams exposed to operational and customer signals?
- Are we optimizing for fast, safe change rather than local team efficiency?

3. Avoid Team Anti-Patterns

- Are we avoiding “throw it over the wall” handoffs between Dev, Test, Ops, and BAU?
- Are we avoiding temporary project teams that disband immediately after delivery?
- Are we avoiding a separate DevOps team that becomes another silo?
- Are we avoiding excessive dependency on specialist teams for routine delivery work?

- Are we avoiding copying another company's model without considering our own context?

4. Choose the Right Topology for Context

- Have we considered organization size and software scale?
- Have we assessed technical and cultural maturity?
- Have we considered engineering discipline, automation, testing, deployment, and monitoring maturity?
- Have we identified which topology fits the current context, not an ideal future state?
- Do we recognize that there is no single "best" topology for every organization?

5. Stream-Aligned / Product Teams

- Are product or stream-aligned teams focused on a clear business stream?
- Can the team design, build, test, deploy, and operate its software?
- Does the team have enough autonomy to deliver without constant blocking dependencies?
- Are dependencies on infrastructure, QA, networking, or operations non-blocking?
- Does the team have access to self-service capabilities where needed?

6. Support Systems and Platforms

- Do product teams have a compatible support system around them?
- Are platform capabilities easy to consume through self-service?
- Are infrastructure, deployment, monitoring, and test environments available without long waits?
- Are platform/cloud teams focused on enabling product teams rather than doing all work for them?
- Are responsibilities split clearly between platform design and application provisioning?

7. Feature Teams

- Do feature teams have high engineering maturity?
- Is there enough trust for teams to change shared code safely?
- Are ownership boundaries clear across components and subsystems?
- Are architecture, system ownership, and integration responsibilities explicitly handled?
- Are teams avoiding shortcuts that create technical debt or reduce trust?

8. SRE and Reliability

- Is SRE being used only where scale and reliability needs justify it?
- Do SRE teams have strong coding and automation skills?
- Are service-level objectives and error budgets defined?
- Is the relationship between SRE and application teams dynamic over time?
- Can application teams take back operational responsibility when appropriate?

9. Dependencies Between Teams

- Have we identified knowledge, task, and resource dependencies?
- Are cross-team dependencies visible and tracked?
- Do we know which dependencies should be removed?
- Do we know which dependencies should be accepted and managed?
- Are dependency counts reviewed before they grow unchecked?

10. Breaking Down Silos

- Have we split responsibilities where doing so reduces bottlenecks?
- Are specialized teams enabling others instead of becoming blockers?
- Are teams empowered to own more of their delivery lifecycle?
- Are shared services designed to reduce friction?
- Are team interactions improving flow rather than reinforcing silos?

11. Evolving the Organization

- Do we treat team topology as something that evolves over time?
- Are topology decisions revisited as maturity, scale, and business needs change?
- Are we using temporary structures only as stepping stones, not permanent silos?
- Do we have a multi-stage approach to improving team design?
- Are team structures optimized for long-term adaptability, not just today's urgent problem?

